

Learning Hugging Face: The 20% That Matters

1 What You Actually Need to Know

The Hugging Face **transformers** library lets you download and run pretrained models in a few lines. The 80% workflow is: pick a model from the Hub, load a **tokenizer** and a **model**, convert text to tensors, run the model, and interpret the output. The single most important skill is knowing the **input and output shapes** — this section focuses heavily on that.

2 Setup

```
1 pip install transformers torch
2 # datasets is useful too:
3 pip install datasets
```

3 The Easiest Path: pipeline()

For quick use, `pipeline` hides all the details. Great for prototyping.

```
1 from transformers import pipeline
2
3 clf = pipeline("sentiment-analysis")
4 print(clf("I love this!"))
5 # [{'label': 'POSITIVE', 'score': 0.9998}]
6
7 # Common tasks (the string sets the behaviour):
8 pipeline("text-classification")
9 pipeline("text-generation", model="gpt2")
10 pipeline("question-answering")
11 pipeline("summarization")
12 pipeline("translation_en_to_fr")
13 pipeline("fill-mask")
14 pipeline("zero-shot-classification")
```

4 The Real Workflow: Tokenizer + Model

This is what you must understand. Two objects, always paired and from the same checkpoint name.

```
1 from transformers import AutoTokenizer, AutoModel
2
3 name = "bert-base-uncased"
```

```

4 tokenizer = AutoTokenizer.from_pretrained(name)
5 model = AutoModel.from_pretrained(name)

```

5 Tokenizer: Inputs and Outputs (Critical)

The tokenizer turns text into integer IDs the model can read.

```

1 inputs = tokenizer("Hello world", return_tensors="pt")
2 print(inputs)
3 # {
4 #   'input_ids':      tensor([[101, 7592, 2088, 102]]),   shape (batch,
5 #   'attention_mask': tensor([[1, 1, 1, 1]]),             shape (batch,
6 #   }

```

What each field means:

- `input_ids` — token IDs, shape (batch_size, seq_len).
- `attention_mask` — 1 for real tokens, 0 for padding, same shape.
- `token_type_ids` — (some models) marks sentence A vs B.

Batching with padding (so all sequences have equal length):

```

1 batch = tokenizer(
2     ["short text", "a much longer piece of text here"],
3     padding=True,           # pad to longest in batch
4     truncation=True,       # cut off at max length
5     max_length=128,
6     return_tensors="pt"    # "pt"=PyTorch, "tf"=TensorFlow, "np"=NumPy
7 )
8 # input_ids shape -> (2, seq_len_of_longest)

```

6 Running the Model: Output Shapes

```

1 import torch
2
3 with torch.no_grad():
4     outputs = model(**inputs)    # ** unpacks the dict as kwargs
5
6 # AutoModel (base) gives hidden states:
7 outputs.last_hidden_state.shape
8 # (batch_size, seq_len, hidden_size) e.g. (1, 4, 768)
9 outputs.pooler_output.shape
10 # (batch_size, hidden_size) -- a summary vector for the sequence

```

7 Task-Specific Model Classes

AutoModel gives raw embeddings. For actual tasks, use the right head, which changes the output shape:

```
1 from transformers import AutoModelForSequenceClassification
2
3 model = AutoModelForSequenceClassification.from_pretrained(
4     "distilbert-base-uncased-finetuned-sst-2-english"
5 )
6 outputs = model(**inputs)
7 outputs.logits.shape           # (batch_size, num_labels)
8
9 # Convert logits -> probabilities -> predicted class
10 probs = torch.softmax(outputs.logits, dim=-1)
11 pred = probs.argmax(dim=-1)    # index of highest-prob class
12 label = model.config.id2label[pred.item()] # human-readable label
```

The Auto classes and their output:

- AutoModel → last_hidden_state (raw features).
- AutoModelForSequenceClassification → logits (batch, num_labels).
- AutoModelForTokenClassification → logits (batch, seq_len, num_labels).
- AutoModelForQuestionAnswering → start_logits, end_logits.
- AutoModelForCausalLM → logits (batch, seq_len, vocab_size) for generation.
- AutoModelForSeq2SeqLM → for translation/summarization.

8 Text Generation

```
1 from transformers import AutoTokenizer, AutoModelForCausalLM
2
3 tok = AutoTokenizer.from_pretrained("gpt2")
4 model = AutoModelForCausalLM.from_pretrained("gpt2")
5
6 inputs = tok("The future of AI is", return_tensors="pt")
7 out_ids = model.generate(
8     **inputs,
9     max_new_tokens=50,
10    do_sample=True,
11    temperature=0.7,
12    top_p=0.9
13 )
14 print(tok.decode(out_ids[0], skip_special_tokens=True))
```

decode reverses tokenization: IDs back to text. skip_special_tokens drops markers like [CLS], [SEP].

9 How to Read the Model Card / Docs

On a model's Hub page (huggingface.co/<name>), look for:

1. The **task tag** (e.g. text-classification) — tells you which Auto class to use.
2. The **usage example** — copy its exact tokenizer + model class.
3. `config.json` — contains `id2label`, `hidden_size`, `max_position_embeddings`.
4. Input expectations — max sequence length, expected language, image size for vision models.

10 Useful Extras

```
1 # Inspect any model's config to learn its shapes
2 print(model.config)
3
4 # Move to GPU
5 model = model.to("cuda")
6 inputs = {k: v.to("cuda") for k, v in inputs.items()}
7
8 # Save / load locally
9 model.save_pretrained("./my_model")
10 tokenizer.save_pretrained("./my_model")
```

11 The Mental Checklist for Any Model

1. What task is it? → picks the Auto class.
2. What does the tokenizer output? → `input_ids` + `attention_mask`, shape (batch, seq).
3. What does the model output? → usually `logits`; check the last dim.
4. How do I turn output into an answer? → `softmax/argmax` + `id2label`, or `decode`.

12 Practice Task

Pick a sentiment model from the Hub. Load its tokenizer and the correct `AutoModelFor...` class. Tokenize a sentence, print the input shapes, run the model, print `logits.shape`, then convert to a human-readable label using `id2label`.